

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

...với quy trình đánh giá chống rò rỉ dữ liệu

Nguyễn Vũ Thái¹ Bùi Văn Dũng² Đỗ Trí Nhựt² Nguyễn Văn Dũ³
FAIR'2026

¹Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam

²Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam

³Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

...với quy trình đánh giá chống rò rỉ dữ liệu

Nguyễn Vũ Thái¹ Bùi Văn Dũng² Đỗ Trí Nhựt² Nguyễn Văn Dũ³
FAIR'2026

¹Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam

²Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam

³Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

Nói: Kính chào quý thầy cô và các anh chị. Tôi là Nguyễn Vũ Thái, xin trình bày bài báo về giảm đặc trưng cho phát hiện xâm nhập mạng trên Spark ML. Cả bài xoay quanh một ý: trên CICIDS2017, điểm F1 gần 1 thường phản ánh cách đánh giá hơn là chất lượng mô hình.

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Vì sao điểm F1 trên 0,99 chưa chắc đáng tin.

II. QUY TRÌNH ĐÁNH GIÁ

Làm sạch dữ liệu và chặn rò rỉ ở từng bước.

III. KẾT QUẢ VÀ KẾT LUẬN

Sáu kết quả, từ giảm đặc trưng đến chuyển sang mạng khác.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

└─Nội dung trình bày

Nói: Bài trình bày gồm ba phần: vấn đề và đóng góp, quy trình đánh giá, rồi sáu kết quả và kết luận. Phần kết quả chiếm nhiều thời gian nhất.

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Vì sao điểm F1 trên 0,99 chưa chắc đáng tin.

II. QUY TRÌNH ĐÁNH GIÁ

Làm sạch dữ liệu và chặn rò rỉ ở từng bước.

III. KẾT QUẢ VÀ KẾT LUẬN

Sáu kết quả, từ giảm đặc trưng đến chuyển sang mạng khác.

I. VẤN ĐỀ VÀ ĐÓNG GÓP

F1 trên 0,99 với CICIDS2017: điểm đó có đáng tin không?

Hai câu hỏi

1. Cần cả ~80 đặc trưng không?

Nhiều đặc trưng thì tốn tính toán, khó chạy trên thiết bị biên.

2. Điểm cao có đáng tin không?

Hai dạng rò rỉ dữ liệu làm điểm cao hơn thực tế:

- Cổng đích: mỗi loại tấn công nhắm vào một số cổng cố định, nên **cổng gần như tiết lộ nhãn**.
- Luồng trùng lặp ở cả hai tập: mô hình **đã thấy dữ liệu kiểm tra khi huấn luyện**.

Bài báo này

Sửa quy trình đánh giá *trước*, rồi mới so sánh các phương pháp giảm đặc trưng.

Thông điệp

Đánh giá đúng cách quan trọng không kém *chọn đúng phương pháp*.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Nói: Bài báo xuất phát từ hai câu hỏi. Thứ nhất, mỗi luồng mạng có khoảng 80 đặc trưng, có cần dùng hết không, vì càng nhiều thì càng tốn và càng khó chạy trên thiết bị biên. Thứ hai, rất nhiều bài đạt F1 trên 0,99 với bộ dữ liệu này, điểm đó có đáng tin không. Có hai dạng rò rỉ dữ liệu khiến điểm cao hơn thực tế. Rò rỉ dữ liệu, nói đơn giản, là mô hình được biết trước thứ mà khi triển khai thật nó không có. Một là cổng đích: trong bộ dữ liệu, mỗi loại tấn công nhắm vào một số cổng cố định, nên cổng gần như tiết lộ nhãn. Hai là luồng trùng nhau nằm ở cả tập huấn luyện lẫn tập kiểm tra, mô hình xem như đã thấy dữ liệu kiểm tra từ lúc huấn luyện. Vì vậy chúng tôi sửa quy trình đánh giá trước, rồi mới so sánh các phương pháp giảm đặc trưng.

Nếu bị hỏi: Rò rỉ dữ liệu là khi thông tin không có lúc triển khai lọt vào huấn luyện hoặc đánh giá. Cột cổng đích trong dữ liệu là destination_port.

Hai câu hỏi

1. Cần cả ~80 đặc trưng không?

Nhiều đặc trưng thì tốn tính toán, khó chạy trên thiết bị biên.

2. Điểm cao có đáng tin không?

Hai dạng rò rỉ dữ liệu làm điểm cao hơn thực tế:

- Cổng đích: mỗi loại tấn công nhắm vào một số cổng cố định, nên **cổng gần như tiết lộ nhãn**.
- Luồng trùng lặp ở cả hai tập: mô hình **đã thấy dữ liệu kiểm tra khi huấn luyện**.

Bài báo này

Sửa quy trình đánh giá trước, rồi mới so sánh các phương pháp giảm đặc trưng.

Thông điệp

Đánh giá đúng cách quan trọng không kém chọn đúng phương pháp.

- 1. So sánh **4 phương pháp giảm đặc trưng** (giữ toàn bộ, RF Importance, SHAP, PCA) trên cùng 8 thuật toán Spark ML.
- 2. **Quy trình đánh giá chống rò rỉ**: bỏ cổng đích, bỏ luồng trùng lặp trước khi chia dữ liệu, tập kiểm tra chỉ dùng một lần ở cuối.
- 3. **Kiểm định thống kê** để kết luận, thay vì chỉ so sánh điểm số.
- 4. Đánh giá hai khía cạnh mà điểm F1 tổng che khuất: **từng loại tấn công** và **đổi sang bộ dữ liệu khác**.
- 5. Chọn cấu hình **phù hợp thiết bị biên** Jetson Orin Nano Super.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

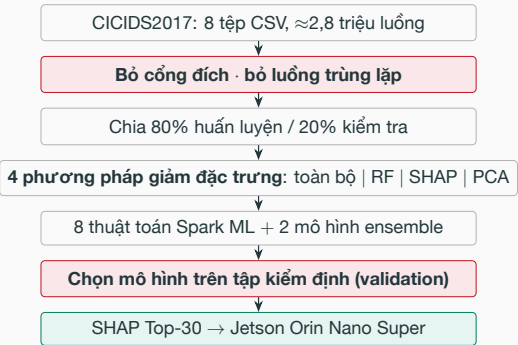
I. VẤN ĐỀ VÀ ĐÓNG GÓP

Nói: Bài báo có năm đóng góp. Một, so sánh bốn phương pháp giảm đặc trưng trên cùng tám thuật toán, cùng một quy trình. Hai, quy trình đánh giá chống rò rỉ. Ba, dùng kiểm định thống kê để kết luận, không chỉ so sánh điểm số. Bốn, đánh giá những gì điểm tổng thường che khuất: từng loại tấn công, và khi đổi sang bộ dữ liệu khác. Năm, chọn cấu hình phù hợp thiết bị biên. Điểm mới là mục hai đến bốn ít khi được làm cùng nhau trong các bài về CICIDS2017.

- 1. So sánh **4 phương pháp giảm đặc trưng** (giữ toàn bộ, RF Importance, SHAP, PCA) trên cùng 8 thuật toán Spark ML.
- 2. **Quy trình đánh giá chống rò rỉ**: bỏ cổng đích, bỏ luồng trùng lặp trước khi chia dữ liệu, tập kiểm tra chỉ dùng một lần ở cuối.
- 3. **Kiểm định thống kê** để kết luận, thay vì chỉ so sánh điểm số.
- 4. Đánh giá hai khía cạnh mà điểm F1 tổng che khuất: **từng loại tấn công** và **đổi sang bộ dữ liệu khác**.
- 5. Chọn cấu hình **phù hợp thiết bị biên** Jetson Orin Nano Super.

II. QUY TRÌNH ĐÁNH GIÁ

Quy trình: mỗi bước chặn một đường rò rỉ



- Bỏ cổng đích: mô hình không thể **suy ra loại tấn công từ số cổng**.
- Bỏ trùng lặp *trước* khi chia: không luồng nào ở cả hai tập.
- Xếp hạng đặc trưng chỉ tính trên *tập huấn luyện*.
- Chọn mô hình trên *tập kiểm định* (validation). Tập kiểm tra **chỉ dùng một lần**, ở cuối.
- Siêu tham số **cố định từ trước**, dùng chung cả 4 phương pháp.

2026-10-08

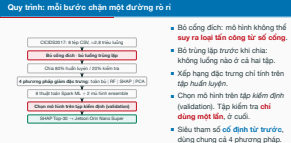
Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

II. QUY TRÌNH ĐÁNH GIÁ

Quy trình: mỗi bước chặn một đường rò rỉ

Nói: Đây là quy trình, đi từ trên xuống. Hai khối đỏ là hai bước chặn rò rỉ. Trước khi chia dữ liệu, chúng tôi bỏ cổng và bỏ luồng trùng, nên không luồng nào nằm ở cả hai tập. Sau đó áp dụng bốn phương pháp giảm đặc trưng, huấn luyện tám thuật toán. Bảng xếp hạng đặc trưng chỉ tính trên tập huấn luyện. Chọn mô hình thì dùng tập kiểm định, tức tập validation tách từ tập huấn luyện. Tập kiểm tra chỉ dùng một lần, ở cuối. Siêu tham số được cố định từ trước và dùng chung cho cả bốn phương pháp, nên khác biệt là do phương pháp chứ không do tinh chỉnh. Cuối cùng, cấu hình SHAP 30 đặc trưng được đưa xuống thiết bị biên.

Nếu bị hỏi: Tập kiểm định tách 20% từ tập huấn luyện. Bốn cấu hình so sánh được cố định từ trước (toàn bộ 77, RF Top-30, SHAP Top-30, PCA $k=40$), siêu tham số dùng chung, nên khác biệt phản ánh phương pháp chứ không phải việc tinh chỉnh.



- Bỏ cổng đích: mô hình không thể **suy ra loại tấn công từ số cổng**.
- Bỏ trùng lặp trước khi chia: không luồng nào ở cả hai tập.
- Xếp hạng đặc trưng chỉ tính trên tập huấn luyện.
- Chọn mô hình trên tập kiểm định (validation). Tập kiểm tra **chỉ dùng một lần**, ở cuối.
- Siêu tham số **cố định từ trước**, dùng chung cả 4 phương pháp.

Loại bỏ luồng trùng lặp: phải làm đúng cách

- Nhiều luồng **giống hệt nhau** ở mọi đặc trưng → chỉ giữ lại *một* luồng.
- Có những nhóm giống hệt nhau nhưng **nhãn khác nhau** (vừa Benign vừa Attack): lỗi đã biết của CICIDS2017.
- Nhóm như vậy được **bỏ cả nhóm: 270 nhóm, 44.260 luồng**.
- Nếu giữ ngẫu nhiên một luồng thì *mỗi lần chạy cho một bộ dữ liệu khác*. Bỏ cả nhóm thì kết quả tái lập được.

Dữ liệu sau khi làm sạch	
Đặc trưng	77
Luồng huấn luyện	1,785,349
Luồng kiểm tra	447,275
CSE-CIC-IDS2018 lấy mẫu 20%	3,239,624

Huấn luyện trên cum Spark gồm **2 Jetson Orin Nano Super** máy chủ chỉ điều phối

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

II. QUY TRÌNH ĐÁNH GIÁ

Nói: Bỏ luồng trùng nghe đơn giản nhưng dễ làm sai. Những luồng giống hệt nhau ở mọi đặc trưng thì chỉ giữ một bản. Nhưng có những nhóm giống hệt nhau mà nhãn lại khác nhau, vừa bình thường vừa tấn công. Đây là lỗi đã biết của CICIDS2017. Với nhóm như vậy chúng tôi bỏ cả nhóm: 270 nhóm, hơn 44 nghìn luồng. Lý do là nếu giữ ngẫu nhiên một luồng thì mỗi lần chạy ra một bộ dữ liệu khác. Sau khi làm sạch còn 77 đặc trưng, khoảng 1,8 triệu luồng huấn luyện và 450 nghìn luồng kiểm tra.

Nếu bị hỏi: Cột bị bỏ: destination_port, source_port, protocol và các cột định danh. Lỗi nhãn mâu thuẫn đã được Lanvin et al. ghi nhận. CSE-CIC-IDS2018 lấy mẫu phân tầng 20%, được 3,14 triệu luồng, còn 2,39 triệu sau khi bỏ trùng. Spark 3.4.1. Máy chủ chỉ làm master, không chạy executor, nên thời gian huấn luyện là thời gian trên hai Jetson.

Loại bỏ luồng trùng lặp: phải làm đúng cách

- Nhiều luồng giống hệt nhau ở mọi đặc trưng → chỉ giữ lại một luồng.
- Có những nhóm giống hệt nhau nhưng **nhãn khác nhau** (vừa Benign vừa Attack): lỗi đã biết của CICIDS2017.
- Nhóm như vậy được **bỏ cả nhóm: 270 nhóm, 44.260 luồng**.
- Nếu giữ ngẫu nhiên một luồng thì mỗi lần chạy cho một bộ dữ liệu khác. Bỏ cả nhóm thì kết quả tái lập được.

Dữ liệu sau khi làm sạch	
Đặc trưng	77
Luồng huấn luyện	1,785,349
Luồng kiểm tra	447,275
CSE-CIC-IDS2018 (lấy mẫu 20%)	3,239,624

Huấn luyện trên cụm Spark gồm 2 **Jetson Orin Nano Super**, máy chủ chỉ điều phối.

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 1: bỏ 60% đặc trưng, F1 gần như không đổi

Phương pháp	Đặc trưng	F1	Phút
Tổng bộ	77	0,9971	77
SHAP Top-30	30	0,9970	20
PCA k=40	40	0,9939	24
RF Top-30	30	0,9922	22

XGBoost, một lần chạy. Phút = thời gian huấn luyện.

30/77 đặc trưng giữ lại
F1 0,9971 → 0,9970, gần như không đổi.

3,8× huấn luyện nhanh hơn
77 phút → 20 phút.

SHAP > RF khi cùng chọn 30
0,9970 so với 0,9922. PCA 0,9939, không giải thích được.

Chọn SHAP Top-30: ít đặc trưng, huấn luyện nhanh, giải thích được.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML
III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 1: bỏ 60% đặc trưng, F1 gần như không đổi.

Nói: Kết quả đầu tiên, trên XGBoost. Ba câu hỏi. Một, bỏ 47 trên 77 đặc trưng thì F1 có giảm không? Gần như không: 0,9971 xuống 0,9970. Hai, huấn luyện có nhanh hơn không? Có, từ 77 phút còn 20 phút, nhanh gấp 3,8 lần. Ba, chọn đặc trưng cách nào tốt nhất? SHAP: cùng 30 đặc trưng, SHAP được 0,9970, RF chỉ 0,9922. PCA cần 40 thành phần, điểm thấp hơn và không giải thích được. Vậy chúng tôi chọn SHAP Top-30.

Nếu bị hỏi: SHAP hơn RF ở cả bốn thuật toán cây, xem slide dự phòng. Thời gian huấn luyện chính xác là 4615,5 s và 1226,1 s, giảm khoảng 73%. PCA dùng k=40 vì nó trích xuất chứ không chọn lọc: mỗi thành phần chỉ mang một phần phương sai.

Kết quả 1: bỏ 60% đặc trưng, F1 gần như không đổi.			
Phương pháp	Đặc trưng	F1	Phút
Tổng bộ	77	0,9971	77
SHAP Top-30	30	0,9970	20
PCA k=40	40	0,9939	24
RF Top-30	30	0,9922	22
XGBoost, một lần chạy. Phút = thời gian huấn luyện.			
Chọn SHAP Top-30: ít đặc trưng, huấn luyện nhanh, giải thích được.			

30/77 đặc trưng giữ lại
F1 0,9971 → 0,9970, gần như không đổi.
3,8× huấn luyện nhanh hơn
77 phút → 20 phút.
SHAP > RF khi cùng chọn 30
0,9970 so với 0,9922. PCA 0,9939, không giải thích được.

Kết quả 2: cổng đích làm tăng điểm, mức tăng tùy mô hình

Nhóm mô hình	F1 tăng khi giữ cổng
Cây (DT, RF, GBT, XGBoost)	dưới 0,002
MLP, Naive Bayes	0,005 và 0,011
Tuyến tính (LR, SVM)	khoảng 0,04

Mỗi thuật toán huấn luyện hai lần trên cổng một cách chia dữ liệu: một lần bỏ cổng đích, một lần giữ.

<0,002 mô hình cây
Hầu như không dựa vào cổng đích.

≈0,04 mô hình tuyến tính
Dựa nhiều: gấp hơn 20 lần mô hình cây.

0,001 nếu chỉ đo trên RF
Trông như không đáng kể, nhưng LR, SVM tăng 0,04.

Rò rỉ phải đo trên *từng* mô hình. Mọi kết quả sau đây đều đã bỏ cổng đích.

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 2: cổng đích làm tăng điểm, mức tăng tùy mô hình

Nói: Kết quả thứ hai: cổng đích làm tăng điểm bao nhiêu. Mỗi thuật toán được huấn luyện hai lần, bỏ cổng và giữ cổng. Một, mô hình cây có dựa vào cổng không? Hầu như không, F1 tăng chưa tới 0,002. Hai, mô hình tuyến tính thì sao? Dựa nhiều, tăng khoảng 0,04, gấp hơn 20 lần. Ba, chỉ đo rò rỉ trên Random Forest có đủ không? Không. Bản nộp đầu chỉ đo trên Random Forest: F1 tăng 0,001, trông như cổng đích không ảnh hưởng gì. Nhưng LR và SVM tăng 0,04. Nên rò rỉ phải đo trên từng mô hình. Từ đây trở đi, mọi kết quả đều đã bỏ cổng.

Nếu bị hỏi: Số từng thuật toán ở slide dự phòng. Mô hình cây dựng lại được cùng ranh giới từ thống kê hành vi của luồng. Mô hình tuyến tính không làm được, nên dựa vào cổng như một định danh dịch vụ. Bản nộp ban đầu chỉ đo trên Random Forest (0,9964 so với 0,9954). Bản camera-ready đo cả tám thuật toán theo góp ý phản biện.

Kết quả 2: cổng đích làm tăng điểm, mức tăng tùy mô hình	
Nhóm mô hình	F1 tăng khi giữ cổng
Cây (DT, RF, GBT, XGBoost)	dưới 0,002
MLP, Naive Bayes	0,005 và 0,011
Tuyến tính (LR, SVM)	khoảng 0,04
Mỗi thuật toán huấn luyện hai lần trên cổng một cách chia dữ liệu: một lần bỏ cổng đích, một lần giữ.	
Rò rỉ phải đo trên <i>từng</i> mô hình. Mọi kết quả sau đây đều đã bỏ cổng đích.	

<0,002 mô hình cây

Hầu như không dựa vào cổng đích.

≈0,04 mô hình tuyến tính

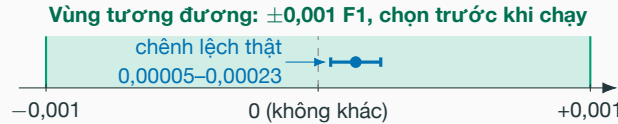
Dựa nhiều: gấp hơn 20 lần mô hình cây.

0,001 nếu chỉ đo trên RF

Trông như không đáng kể, nhưng LR, SVM tăng 0,04.

Kết quả 3: 77 hay 30 đặc trưng, F1 tương đương về mặt thống kê

XGBboost, N=6 lần chạy	F1 trung bình
Tập bộ 77 đặc trưng	0,99734
SHAP Top-30	0,99720
Chênh lệch	0,0001



$\pm 0,00012$ sai số

77 đặc trưng; SHAP Top-30
 $\pm 0,00015$ (CI95
Nadeau–Bengio).

$p=0,031$ có khác

Nhưng chỉ $\approx 0,0001$. $d \approx 1,94$
vì 6 lần chạy rất sát (hoán vị
ghép cặp).

$p=3,9 \times 10^{-6}$

Tương đương (TOST, biên
 $\pm 0,001$ F1).

30 đặc trưng tốt ngang 77, nên chọn **SHAP Top-30**: gọn hơn, giải thích được.

2026-10-08

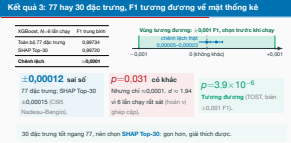
Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 3: 77 hay 30 đặc trưng, F1 tương đương về mặt

Nói: Kết quả 1 chỉ là một lần chạy, nên chúng tôi chạy lại 6 lần. F1 hai bên chênh khoảng 0,0001. Chúng tôi trả lời ba câu hỏi. Một, số đo có chắc không? Chắc, sai số chỉ khoảng cộng trừ 0,0001. Hai, hai bên có khác nhau không? Có, nhưng rất nhỏ. Ba, có coi như nhau được không? Được. Nhìn hình: vùng xanh lá là mức chênh cộng trừ 0,001 mà chúng tôi chấp nhận, đặt trước khi chạy. Đoạn xanh dương là chênh lệch thật, nằm gọn trong đó. Vì vậy chọn SHAP Top-30: ít đặc trưng hơn mà tốt ngang, lại giải thích được.

Nếu bị hỏi: Phép 1: các lần chạy dùng chung dữ liệu nên khoảng tin cậy thông thường hẹp quá, Nadeau–Bengio hiệu chỉnh việc đó. Phép 2: kiểm định hoán vị hai phía liệt kê đủ 2^6 tổ hợp dấu, nên p nhỏ nhất là $2/2^6 = 0,031$, cần $N \geq 6$. Cohen's $d \approx 1,94$ lớn vì phương sai giữa các lần chạy rất nhỏ, không phải vì chênh lệch lớn. Phép 3: p của phép 2 không chứng minh được hai bên giống nhau, nên dùng TOST với biên $\pm 0,001$ F1 trên sai số Nadeau–Bengio; biên nhỏ nhất chứng minh được ở $\alpha = 0,05$ là $\pm 2,3 \times 10^{-4}$. Xem slide phụ thuộc.



Kết quả 4: điểm tổng 0,998 che khuất các loại tấn công hiểm

Loại lưu lượng	Số mẫu	Tỉ lệ phát hiện
Benign	380.119	99,97%
DoS Hulk	34.446	99,04%
Web – Brute Force	311	72,99%
Bot	290	45,52%
Web – XSS	111	2,70%
Web – SQL Injection	6	0%
Random Forest, 77 đặc trưng, Tỉ lệ phát hiện = recall		

Lớp càng ít mẫu càng dễ bị bỏ sót, và điểm tổng không phản ánh điều đó.

0,998 → 0,806

Khi tính đều từng lớp: điểm tổng che khuất lớp hiểm.

6/6 SQL Injection bị bỏ sót

Cùng 54% Bot bị xếp vào bình thường.

76% XSS vẫn bị báo là tấn công

Chỉ nhầm loại, không bỏ sót.

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Nói: Đến đây mới chỉ hỏi có tấn công hay không. Giờ hỏi là loại tấn công nào. Một, điểm tổng có phản ánh đúng không? Không: điểm chung là 0,998, nhưng tính đều cho từng lớp thì chỉ còn 0,806. Hai, lớp nào bị bỏ sót? Các lớp hiểm: cả 6 mẫu SQL Injection và 54% Bot bị xếp là bình thường. Ba, XSS chỉ phát hiện đúng 2,7%, có phải bỏ sót không? Phần lớn không: 76% XSS vẫn bị báo là tấn công, chỉ nhầm sang loại khác. Bỏ sót mới là lỗi nghiêm trọng.

Nếu bị hỏi: Tỉ lệ phát hiện là recall. Mô hình là Random Forest dùng đủ 77 đặc trưng. So sánh các cách giảm đặc trưng ở chế độ đa lớp là hướng tiếp theo. 0,998 là weighted-F1, 0,806 là macro-F1. Ma trận nhầm lẫn ở slide dự phòng. XSS: 81/111 mẫu bị dự đoán là Web Brute Force. Bot 158/290 và 27% Brute Force bị dự đoán là Benign. Nguyên nhân có thể: mất cân bằng cực độ (6–311 mẫu so với 380 nghìn Benign), bỏ cổng rồi thì luồng HTTP ngắn của tấn công web giống lưu lượng duyệt web thông thường, và Bot C&C giống lưu lượng an ninh.



Kết quả 5: đổi sang bộ dữ liệu khác, điểm số giảm mạnh

Huấn luyện → Kiểm tra	F1	AUC-PR
2017 → 2017	0,995	1,00
2017 → 2018	0,03–0,30	0,48
2018 → 2018	0,925	0,97
2018 → 2017	0,05	0,19

2017 → CICIDS2017 2017 → 2018 → CICIDS2018

0,995 → 0,03–0,30

Chiều ngược lại: 0,925 → 0,05.

Điểm cao trên một bộ dữ liệu **không đảm bảo** hoạt động tốt trên mạng khác.



7% / 3% tấn công bắt được

Mô hình bỏ sót gần hết.

11 + 7 lần chạy, hai cụm F1

Một lần chạy không đủ tin: báo cả 18 lần, so thêm bằng **AUC-PR**.

2026-10-08

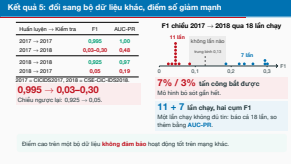
Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 5: đổi sang bộ dữ liệu khác, điểm số giảm mạnh

Nói: Đây là kết quả quan trọng nhất. Một, sang mạng khác mô hình còn tốt không? Không: cùng bộ thì F1 là 0,995, sang bộ 2018 chỉ còn 0,03 đến 0,30. Chiều ngược lại, 0,925 còn 0,05. Hai, mô hình sai kiểu gì? Bỏ sót: chỉ phát hiện được 7% và 3% số tấn công. Ba, một lần chạy có đủ tin không? Không. Hình bên phải: 18 lần chạy cùng mã, cùng dữ liệu, F1 rơi vào hai cụm, 11 lần thấp, 7 lần cao, không lần nào ở giữa. Nên chúng tôi báo cả 18 lần và so sánh thêm bằng AUC-PR, là điểm không phụ thuộc ngưỡng nên ổn định hơn F1.

Nếu bị hỏi: AUC-PR không phụ thuộc ngưỡng, giảm từ 0,9997 xuống 0,4849 (trung bình 18 lần) và từ 0,9679 xuống 0,1926. Recall 0,073 / 0,030, precision 0,772 / 0,272: mô hình bỏ sót chứ không báo động giả nhiều. Hai cụm có AUC-PR như nhau (Welch $|t| = 1,3$): thứ hạng không đổi, chỉ có ngưỡng 0,5 cắt qua một cụm luồng gần giống nhau. Năm seed chạy lại, hai lần đổi cụm. Nguyên nhân dịch chuyển: khác phiên bản CICFlowMeter, cấu hình mạng, công cụ tấn công. Xem slide dự phòng.



Kết quả 6: gán nhãn 1% dữ liệu mạng mới rồi huấn luyện lại là đủ

Cách làm (F1)	2017 → 2018	2018 → 2017
Dùng mô hình cũ	0,03-0,30	0,05
Chỉnh phân phối (CORAL)	0	0
Huấn luyện lại trên 1%	0,92	0,99
1% dữ liệu cũ	0,80	0,96

Mạng mới = bỏ dữ liệu còn lại (2017 ↔ 2018), 1% = 1% dữ liệu cũ/mới của mạng mới, khoảng 19 nghìn luồng

F1 = 0 khi chỉ chỉnh phân phối
Không nhãn (CORAL): khác biệt nằm ở hành vi tấn công.

1% dữ liệu có nhãn là đủ
≈19 nghìn luồng: F1 lên 0,92 và 0,99.

0,80 / 0,96 khi gộp dữ liệu cũ
Kém hơn chỉ dùng 1% của mạng mới.

Sang mạng mới, cách rẻ nhất: **gán nhãn 1%** lưu lượng của chính mạng đó rồi huấn luyện lại.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Nói: Vậy làm sao cho mô hình chạy được trên mạng mới? Một, chỉ chỉnh phân phối dữ liệu cho giống dữ liệu cũ, không cần nhãn, có được không? Không: F1 về 0 ở cả hai chiều. Hai, cần bao nhiêu dữ liệu có nhãn của mạng mới? Rất ít: 1%, khoảng 19 nghìn luồng, huấn luyện lại thì F1 lên 0,92 và 0,99. Ba, gộp thêm toàn bộ dữ liệu cũ có tốt hơn không? Không, lại kém hơn: 0,80 và 0,96. Vậy cách rẻ nhất là gán nhãn một ít lưu lượng của chính mạng mới.

Nếu bị hỏi: Refit scaler và CORAL không cần nhãn, không huấn luyện lại, và cho F1 bằng 0 ở cả hai chiều, trong cả 3 lần chạy: dịch chuyển nằm ở phân phối có điều kiện, không chỉ ở trung bình và hiệp phương sai. CORAL vẫn nâng AUC-PR chiều 2018→2017 từ 0,179 lên 0,250: thứ hạng tốt lên nhưng ngưỡng 0,5 học từ dữ liệu cũ không còn đúng, nên thích ứng phải chỉnh cả ngưỡng. 1% là 19.276 luồng (chiều ngược lại 18.011). Trong thí nghiệm, nhãn lấy từ nhãn có sẵn của bộ dữ liệu, lấy 1% tập huấn luyện của bộ đích, mô hình mới chuyên viên gán nhãn, mô hình được huấn luyện lại thì điều chỉnh phân

Kết quả 8: gán nhãn 1% dữ liệu mạng mới rồi huấn luyện lại là đủ

Cách làm (F1)	2017 → 2018	2018 → 2017
Dùng mô hình cũ	0,03-0,30	0,05
Chỉnh phân phối (CORAL)	0	0
Huấn luyện lại trên 1%	0,92	0,99
1% dữ liệu cũ	0,80	0,96

Mạng mới = bỏ dữ liệu còn lại (2017 ↔ 2018), 1% = 1% dữ liệu cũ/mới của mạng mới, khoảng 19 nghìn luồng.

Kiểm tra chỉ dùng 1% của mạng mới.

F1 = 0 khi chỉ chỉnh phân phối
Không nhãn (CORAL): khác biệt nằm ở hành vi tấn công.

1% dữ liệu có nhãn là đủ
≈19 nghìn luồng: F1 lên 0,92 và 0,99.

0,80 / 0,96 khi gộp dữ liệu cũ
Kém hơn chỉ dùng 1% của mạng mới.

Sang mạng mới, cách rẻ nhất: **gán nhãn 1%** lưu lượng của chính mạng đó rồi huấn luyện lại.

Kết quả 6: gán nhãn 1% dữ liệu mạng mới rồi huấn luyện lại là đủ

60%

đặc trưng bỏ được

F1 không đổi (đã kiểm định), nhanh 3,8 lần.

0,995 → 0,03–0,30

khi sang mạng khác

Mô hình bỏ sót gần hết tấn công.

0,998 → 0,806

khi tính đều từng loại tấn công

Điểm tổng cao che khuất các tấn công hiếm.

1%

dữ liệu có nhãn của mạng mới là đủ

≈19 nghìn luồng: F1 lên 0,92 và 0,99.

Bài học: **đánh giá đúng cách** quan trọng không kém chọn đúng phương pháp.

Xin cảm ơn. Kính mời câu hỏi

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Nói: Tóm lại có bốn con số cần nhớ. Một, 60%: bỏ được 60% đặc trưng mà F1 vẫn tương đương, huấn luyện nhanh gấp 3,8 lần. Hai, từ 0,998 xuống 0,806: điểm tổng cao che khuất các tấn công hiếm. Ba, từ 0,995 xuống còn 0,03 đến 0,30: sang mạng khác, mô hình bỏ sót gần hết tấn công. Bốn, 1%: chỉ cần 1% dữ liệu có nhãn của mạng mới, khoảng 19 nghìn luồng, để huấn luyện lại. Bài học chung: đánh giá đúng cách quan trọng không kém chọn đúng phương pháp. Xin cảm ơn quý thầy cô, kính mời câu hỏi.

Nếu bị hỏi: Hướng tiếp theo: đo riêng tác động của bước bỏ trùng lặp; toàn bộ CSE-CIC-IDS2018 và các bộ mới hơn (RoEduNet, CIC-IoT); thử bộ 30 đặc trưng SHAP trên bộ dữ liệu khác. Tăng số lần lặp bằng kiểm định chéo lồng cũng là hướng củng cố.

Kết luận

60% đặc trưng bỏ được F1 không đổi (đã kiểm định), nhanh 3,8 lần.	0,998 → 0,806 khi tính đều từng loại tấn công Điểm tổng cao che khuất các tấn công hiếm.
0,995 → 0,03–0,30 khi sang mạng khác Mô hình bỏ sót gần hết tấn công.	1% dữ liệu có nhãn của mạng mới là đủ ≈19 nghìn luồng: F1 lên 0,92 và 0,99.

Bài học: **đánh giá đúng cách** quan trọng không kém chọn đúng phương pháp.

Xin cảm ơn. Kính mời câu hỏi

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Lanvin, M. et al. *Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes*. CRiSIS 2022. **mâu thuẫn nhãn**

[3] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **rò rỉ**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP**

[5] Nadeau, C. & Bengio, Y. *Inference for the Generalization Error*. Mach. Learn. 52(3), 2003. **khoảng tin cậy**

[6] D'hooge, L. et al. *Inter-dataset generalization strength of supervised machine learning methods for intrusion detection*. J. Information Security and Applications 54, 2020. **liên bộ dữ liệu**

[7] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark ML**

[8] CIC & CSE. *CSE-CIC-IDS2018 dataset*. 2018. **bộ dữ liệu thứ hai**

[9] Schuirmann, D. J. *A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability*. J. Pharmacokinetics and Biopharmaceutics 15(6), 1987. **TOST**

[10] Sun, B., Feng, J. & Saenko, K. *Return of frustratingly easy domain adaptation*. AAAI 2016. **CORAL**

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

III. KẾT QUẢ VÀ KẾT LUẬN

Không trình bày. Để trên màn hình trong lúc nhận câu hỏi, nên nguồn luôn hiện sẵn nếu khán giả hỏi. Danh sách đầy đủ nằm trong bài báo.

2026-10-08

Tài liệu tham khảo chính	
[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSP 2018.	CICIDS2017
[2] Lanvin, M. et al. Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. CRiSIS 2022.	mâu thuẫn nhãn
[3] Arp, D. et al. Dos and Don'ts of Machine Learning in Computer Security. USENIX Security 2022.	rò rỉ
[4] Lundberg, S.M. & Lee, S.-I. A unified approach to interpreting model predictions. NeurIPS 2017.	SHAP
[5] Nadeau, C. & Bengio, Y. Inference for the Generalization Error. Mach. Learn. 52(3), 2003.	khoảng tin cậy
[6] D'hooge, L. et al. Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. J. Information Security and Applications 54, 2020.	liên bộ dữ liệu
[7] Zaharia, M. et al. Spark: Cluster computing with working sets. HotCloud 2010.	Spark ML
[8] CIC & CSE. CSE-CIC-IDS2018 dataset. 2018.	bộ dữ liệu thứ hai
[9] Schuirmann, D. J. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. J. Pharmacokinetics and Biopharmaceutics 15(6), 1987.	TOST
[10] Sun, B., Feng, J. & Saenko, K. Return of frustratingly easy domain adaptation. AAAI 2016.	CORAL

Slide dự phòng

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập
mạng trên Apache Spark ML

Slide dự phòng

Dự phòng: RF và SHAP khi cùng chọn 30 đặc trưng

Thuật toán (F1)	RF Top-30	SHAP Top-30	Chênh lệch
Random Forest	0,9879	0,9955	+0,008
XGBoost	0,9922	0,9970	+0,005
GBT	0,9844	0,9918	+0,007
Decision Tree	0,9835	0,9948	+0,011

SHAP chọn cho F1 cao hơn RF chọn ở cả bốn thuật toán cây.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Dự phòng: RF và SHAP khi cùng chọn 30 đặc trưng

Dự phòng: RF và SHAP khi cùng chọn 30 đặc trưng			
Thuật toán (F1)	RF Top-30	SHAP Top-30	Chênh lệch
Random Forest	0,9879	0,9955	-0,008
XGBoost	0,9922	0,9970	-0,005
GBT	0,9844	0,9918	-0,007
Decision Tree	0,9835	0,9948	-0,011
SHAP chọn cho F1 cao hơn RF chọn ở cả bốn thuật toán cây.			

Dự phòng: cổng đích theo từng thuật toán

Thuật toán	Bỏ cổng đích	Giữ cổng đích	Chênh lệch
SVM (LinearSVC)	0.7859	0.8274	+0,0415
Logistic Regression	0.7986	0.8381	+0,0395
Naive Bayes	0.2985	0.3093	+0.0108
MLP	0.9660	0.9711	+0.0052
GBT	0.9924	0.9941	+0.0017
Decision Tree	0.9953	0.9966	+0.0014
Random Forest	0.9954	0.9964	+0.0010
XGBoost	0.9972	0.9979	+0.0007

F1 nhị phân, hai lượt trên cùng một cách chia dữ liệu.

Dự phòng

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

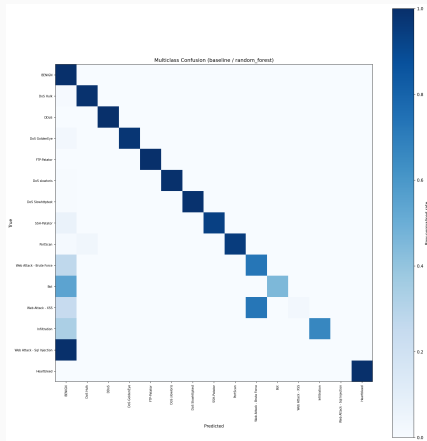
Dự phòng: cổng đích theo từng thuật toán

Dự phòng: cổng đích theo từng thuật toán

Thuật toán	Bỏ cổng đích	Giữ cổng đích	Chênh lệch
SVM (LinearSVC)	0.7859	0.8274	+0,0415
Logistic Regression	0.7986	0.8381	+0,0395
Naive Bayes	0.2985	0.3093	+0.0108
MLP	0.9660	0.9711	+0.0052
GBT	0.9924	0.9941	+0.0017
Decision Tree	0.9953	0.9966	+0.0014
Random Forest	0.9954	0.9964	+0.0010
XGBoost	0.9972	0.9979	+0.0007

F1 nhị phân, hai lượt trên cùng một cách chia dữ liệu.

Dự phòng: ma trận nhầm lẫn theo từng loại tấn công

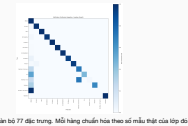


Dự phòng

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

- ↳ Dự phòng: ma trận nhầm lẫn theo từng loại tấn công



Dự phòng: chi tiết kiểm định thống kê

XGBoost	F1 ± CI95 (t)	CI95 (N-B)
Tập bộ 77	0.99734 ± 0.00007	± 0.00012
SHAP Top-30	0.99720 ± 0.00009	± 0.00015
hoán vị ghép cặp $p = 0.031$, Cohen's $d = 1.94$		

TOST (ai kiểm định một phía)		
Đến tương đương, khai báo trước		0.001 F1
Sai số dùng để kiểm định		Nadeau-Bengio
P_{null}		2.9×10^{-6}
Đến nhỏ nhất chứng minh được ($\alpha=0.05$)		$\pm 2.3 \times 10^{-6}$

- $N=6$ lần lấy mẫu lại ghép cặp: phương sai phản ánh việc lấy mẫu dữ liệu, không chỉ seed ngẫu nhiên.
- Hoán vị hai phía, liệt kê đủ 2^6 tổ hợp dấu: p nhỏ nhất là $2/2^6 \approx 0,031$, nên $N \geq 6$ là bắt buộc.
- Các lần lấy mẫu chồng nhau phá vỡ tính độc lập của t , nên dùng CI hiệu chỉnh Nadeau–Bengio.
- Kiểm định ý nghĩa không chứng minh được tương đương. TOST kiểm định trực tiếp điều đó.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Dự phòng: chi tiết kiểm định thống kê

Dự phòng: chi tiết kiểm định thống kê

XGBoost	F1 ± CI95 (t)	CI95 (N-B)
Tập bộ 77	0.99734 ± 0.00007	± 0.00012
SHAP Top-30	0.99720 ± 0.00009	± 0.00015
hoán vị ghép cặp $p = 0.031$, Cohen's $d = 1.94$		

TOST (ai kiểm định một phía)		
Đến tương đương, khai báo trước		0.001 F1
Sai số dùng để kiểm định		Nadeau-Bengio
P_{null}		2.9×10^{-6}
Đến nhỏ nhất chứng minh được ($\alpha=0.05$)		$\pm 2.3 \times 10^{-6}$

- $N=6$ lần lấy mẫu lại ghép cặp: phương sai phản ánh việc lấy mẫu dữ liệu, không chỉ seed ngẫu nhiên.
- Hoán vị hai phía, liệt kê đủ 2^6 tổ hợp dấu: p nhỏ nhất là $2/2^6 \approx 0.031$, nên $N \geq 6$ là bắt buộc.
- Các lần lấy mẫu chồng nhau phá vỡ tính độc lập của t , nên dùng CI hiệu chỉnh Nadeau–Bengio.
- Kiểm định ý nghĩa không chứng minh được tương đương. TOST kiểm định trực tiếp điều đó.

Dự phòng: chi tiết liên bộ dữ liệu

Huấn luyện → Kiểm tra	F1	AUC-PR
2017 → 2017	0.9953	0.9997
2017 → 2018 (11 lần)	0.029-0.085	0.4778
2017 → 2018 (7 lần)	0.185-0.301	0.4960
2018 → 2018	0.9246	0.9679
2018 → 2017	0.0544 ± 0.0019	0.1926

18 lần chạy cùng mã và dữ liệu (13 seed, trong đó 5 seed chạy hai lần).

- Trung bình F1 0,1276 rơi vào khoảng trống, và CI95 của nó chỉ chứa 1/18 lần chạy.
- AUC-PR hai cụm không khác nhau (Welch |t|=1,3): thứ hạng không đổi, chỉ ngưỡng 0,5 cắt qua một cụm luồng gần giống nhau.
- 5 seed chạy lại: 3 giữ cụm, 2 đổi cụm.
- Refit scaler, CORAL: F1 bằng 0 trong cả 3 lần. CORAL nâng AUC-PR 2018→2017 từ 0,179 lên 0,250.

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Dự phòng: chi tiết liên bộ dữ liệu

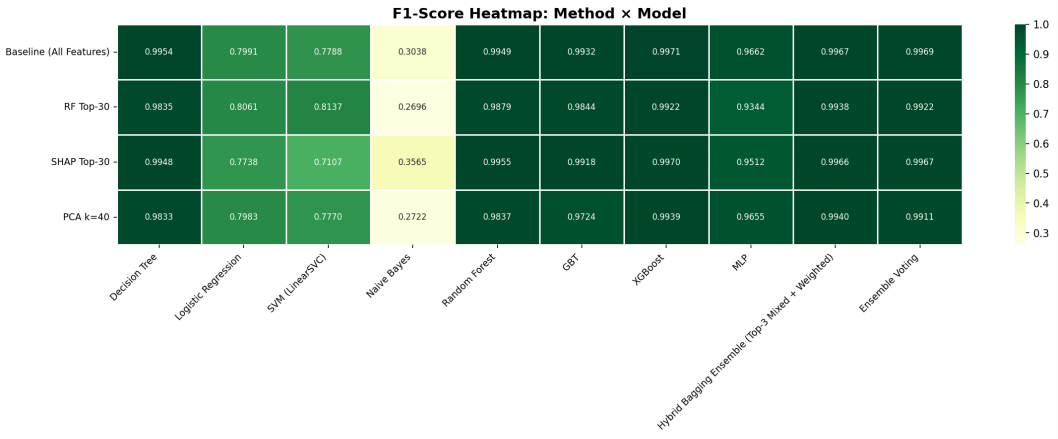
Dự phòng: chi tiết liên bộ dữ liệu

Huấn luyện → Kiểm tra	F1	AUC-PR
2017 → 2017	0.9953	0.9997
2017 → 2018 (11 lần)	0.029-0.085	0.4778
2017 → 2018 (7 lần)	0.185-0.301	0.4960
2018 → 2018	0.9246	0.9679
2018 → 2017	0.0544 ± 0.0019	0.1926

18 lần chạy cùng mã và dữ liệu (13 seed, trong đó 5 seed chạy hai lần).

- Trung bình F1 0,1276 rơi vào khoảng trống, và CI95 của nó chỉ chứa 1/18 lần chạy.
- AUC-PR hai cụm không khác nhau (Welch |t|=1,3): thứ hạng không đổi, chỉ ngưỡng 0,5 cắt qua một cụm luồng gần giống nhau.
- 5 seed chạy lại: 3 giữ cụm, 2 đổi cụm.
- Refit scaler, CORAL: F1 bằng 0 trong cả 3 lần. CORAL nâng AUC-PR 2018 → 2017 từ 0,179 lên 0,250.

Dự phòng: heatmap F1, phương pháp × thuật toán



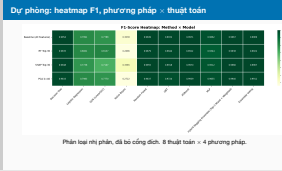
Phân loại nhị phân, đã bỏ cổng dịch, 8 thuật toán × 4 phương pháp.

Dự phòng

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Dự phòng: heatmap F1, phương pháp × thuật toán



Dự phòng: siêu tham số, cố định từ trước

Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Dùng chung cho mọi phương pháp giảm chiều, nên khác biệt phản ánh *phương pháp* chứ không phải việc tinh chỉnh riêng từng mô hình. Trong số lớp cho 6 mô hình, `scale_pos_weight` cho XGBoost. MLP không có trọng số (Spark ML không hỗ trợ trọng số mẫu cho nó, đã nêu trong phần hạn chế. LightGBM bị loại: thư viện gốc qua SynapseML chỉ chạy x86_64, còn đích triển khai là ARM64.

Dự phòng

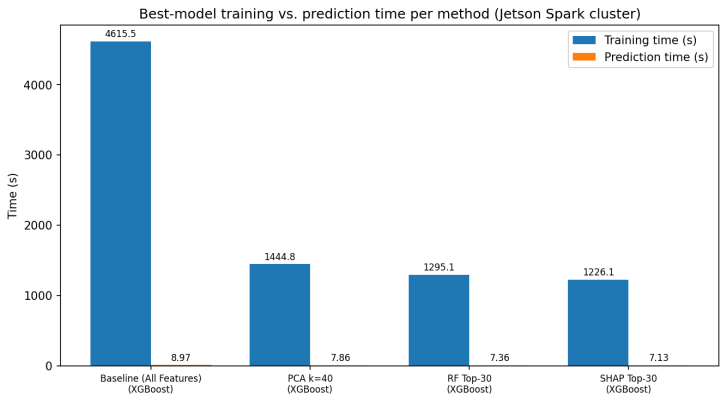
2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

└─Dự phòng: siêu tham số, cố định từ trước

Dự phòng: siêu tham số, cố định từ trước	
Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01
Dùng chung cho mọi phương pháp giảm chiều, nên khác biệt phản ánh phương pháp chứ không phải việc tinh chỉnh riêng từng mô hình. Trong số tập cho 6 mô hình, <code>scale_pos_weight</code> cho XGBoost. MLP không có trọng số (Spark ML không hỗ trợ trọng số mẫu cho nó, đã nêu trong phần hạn chế. LightGBM bị loại: thư viện gốc qua SynapseML chỉ chạy x86_64, còn đích triển khai là ARM64.	

Dự phòng: chi phí huấn luyện và suy luận



Đo trên cụm huấn luyện Spark. Đồ trẻ và thông lượng trên thiết bị biên thuộc bài hệ thống đi kèm.

Dự phòng

2026-10-08

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Dự phòng: chi phí huấn luyện và suy luận



Dự phòng: các yếu tố ảnh hưởng tính hợp lệ

- **Nội tại.** Xếp hạng RF/SHAP lấy từ tập huấn luyện gốc và giữ cố định qua các lần lấy mẫu lại, nhất quán với thiết kế cố định từ trước, nhưng có thể gây rò rỉ nhẹ ở phần xếp hạng. Xếp hạng lại trong hai trong sáu lần lấy mẫu chỉ làm F1 lệch $\leq 7,4 \times 10^{-5}$, và 25–26/30 đặc trưng vẫn được chọn lại. MLP mang hạn chế không có trọng số lớp.
- **Ngoại tại.** CICIDS2017 có thể không phản ánh lưu lượng hiện tại, và các bộ dữ liệu NIDS chuẩn đều có hạn chế chất lượng đã ghi nhận. Kết luận liên bộ dữ liệu dựa trên *mẫu phân tầng* của CSE-CIC-IDS2018, chưa phải toàn bộ. Chưa kiểm thử lưu lượng né tránh. Bản thân việc khử trùng lặp cũng làm thay đổi phân bố benign/attack.
- **Thống kê.** $N=6$ cho kiểm định hoán vị *power* thấp: liệt kê đầy đủ hai phía chặn p ở mức $2/2^N$, nên $N=6$ vừa đủ chạm 0,05. Kết luận tương đương không dựa vào p đó mà dựa vào TOST, trên cùng sai số Nadeau–Bengio đã tính đến việc các lần lấy mẫu chồng nhau. F1 liên bộ dữ liệu không tắt định giữa các lượt chạy, nên được báo trên 18 lượt. Kiểm định chéo lồng với N lớn hơn là hướng củng cố tiếp theo.

2026-10-08

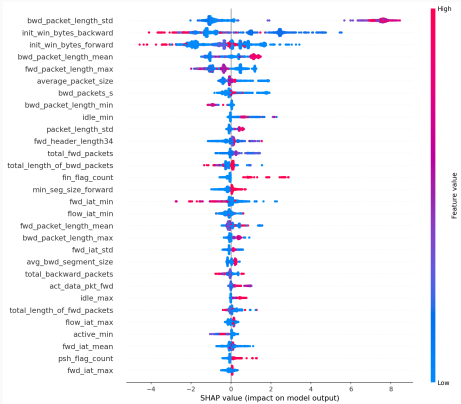
Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

└─ Dự phòng: các yếu tố ảnh hưởng tính hợp lệ

Dự phòng: các yếu tố ảnh hưởng tính hợp lệ

- **Nội tại.** Xếp hạng RF/SHAP lấy từ tập huấn luyện gốc và giữ cố định qua các lần lấy mẫu lại, nhất quán với thiết kế cố định từ trước, nhưng có thể gây rò rỉ nhẹ ở phần xếp hạng. Xếp hạng lại trong hai trong sáu lần lấy mẫu chỉ làm F1 lệch $\leq 7,4 \times 10^{-5}$, và 25–26/30 đặc trưng vẫn được chọn lại. MLP mang hạn chế không có trọng số lớp.
- **Ngoại tại.** CICIDS2017 có thể không phản ánh lưu lượng hiện tại, và các bộ dữ liệu NIDS chuẩn đều có hạn chế chất lượng đã ghi nhận. Kết luận liên bộ dữ liệu dựa trên mẫu phân tầng của CSE-CIC-IDS2018, chưa phải toàn bộ. Chưa kiểm thử lưu lượng né tránh. Bản thân việc khử trùng lặp cũng làm thay đổi phân bố benign/attack.
- **Thống kê.** $N=6$ cho kiểm định hoán vị *power* thấp: liệt kê đầy đủ hai phía chặn p ở mức $2/2^N$, nên $N=6$ vừa đủ chạm 0,05. Kết luận tương đương không dựa vào p đó mà dựa vào TOST, trên cùng sai số Nadeau–Bengio đã tính đến việc các lần lấy mẫu chồng nhau. F1 liên bộ dữ liệu không tắt định giữa các lượt chạy, nên được báo trên 18 lượt. Kiểm định chéo lồng với N lớn hơn là hướng củng cố tiếp theo.

Dự phòng: mức đóng góp của từng đặc trưng theo SHAP



SHAP được tính trên *mẫu phân tầng theo lớp* để cả lưu lượng bình thường và lưu lượng tấn công đều có mặt trong bảng xếp hạng.

Dự phòng

Giảm chiều và lựa chọn đặc trưng cho phát hiện xâm nhập mạng trên Apache Spark ML

Dự phòng: mức đóng góp của từng đặc trưng theo SHAP

